DEPARTMENT OF COMPUTER APPLICATIONS

Amal Jyothi College of Engineering Autonomous, Kanjirappally

SOFTWARE TESTING LAB — AUTOMATED SELENIUM TEST SUITE

Project: MediTwin AI — Intelligent Patient Digital Twin & Healthcare System

---

Test Case 1: Login Test

Code

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestLogin:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()  # Maximize window to prevent hidden elements
        self.wait = WebDriverWait(self.driver, 10)  # Explicit wait

    def teardown_method(self, method):
        self.driver.quit()

    def test_login(self):
        # Navigate to MediTwin AI login portal
        self.driver.get("http://localhost:3000/login")

        # Wait for email field to appear and enter registered email
        email_field = self.wait.until(
            EC.visibility_of_element_located((By.NAME, "email"))
        )
        email_field.send_keys("doctor.rohith@meditwin.local")

        # Wait for password field to appear and enter password
        password_field = self.wait.until(
            EC.visibility_of_element_located((By.NAME, "password"))
        )
        password_field.send_keys("DoctorPass@123")

        # Wait for login button and click it
        login_btn = self.wait.until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']"))
        )
        login_btn.click()

        # Verify successful login and navigation to dashboard
        self.wait.until(EC.url_contains("/dashboard/doctor"))
        time.sleep(2)
```

PowerShell — pytest execution verification

```
PS D:\SEM 2\A.I\MediTwin> pytest tests/selenium/test_case_1_login.py
============================= test session starts ==============================
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\SEM 2\A.I\MediTwin
collected 1 item

test_case_1_login.py .                                                   [100%]

============================== 1 passed in 4.12s ===============================
```

Test Report

Test Case 1

Project Name: MediTwin AI

Login Test Case					
Test Case ID: Test_1			**Test Designed By:** Biya Jomon		
Test Priority(Low/Medium/High): High			**Test Designed Date:** 20/10/2026		
Module Name: Login page			**Test Executed By :** Biya Jomon		
Test Title : verify login with valid username and password.			**Test Execution Date:** 20/10/2026		
Description:	User has valid username and password.				
Pre-Condition : User has valid username and password registered in the MediTwin AI database.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Navigate to the login page	URL: http://localhost:3000/login	Display login page	login page displayed	**pass**
2	Valid email id entered	doctor.rohith@meditwin.local	Field accepts valid email format	Valid email entered	**pass**
3	Valid password entered	DoctorPass@123	Password masked securely	Password entered	**pass**
4	Click on login button	Click Sign In button	User should successfully login and doctor dashboard should be visible	User successfully logged in	**pass**
5	Display Doctor Clinical Dashboard	URL: http://localhost:3000/dashboard/doctor	Doctor Clinical Workstation and telemetry widgets displayed	Doctor Workstation displayed	**pass**
Post-Condition: Username and password is checked with database values for successful login and dashboard session initialization.					

Test Case 2: Registration test of already existing user

Code

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestRegistrationExistingUser:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)

    def teardown_method(self, method):
        self.driver.quit()

    def test_registration_already_existing(self):
        # 1. Navigate to the registration page
        self.driver.get("http://localhost:3000/register")

        # 2. Valid full name entered
        name_field = self.wait.until(
            EC.visibility_of_element_located((By.NAME, "fullName"))
        )
        name_field.clear()
        name_field.send_keys("Dr. Sarah Jenkins")

        # 3. Valid date of birth entered
        dob_field = self.driver.find_element(By.NAME, "dob")
        dob_field.send_keys("1995-06-15")

        # 4. Valid gender selected
        gender_select = self.driver.find_element(By.NAME, "gender")
        gender_select.send_keys("Female")

        # 5. Valid blood group selected
        blood_select = self.driver.find_element(By.NAME, "bloodGroup")
        blood_select.send_keys("O+")

        # 6. Valid phone number entered
        phone_field = self.driver.find_element(By.NAME, "phone")
        phone_field.clear()
        phone_field.send_keys("9876543210")

        # 7. Enter already registered email id
        email_field = self.driver.find_element(By.NAME, "email")
        email_field.clear()
        email_field.send_keys("sarah.jenkins@meditwin.local")

        # 8. Enter username
        username_field = self.driver.find_element(By.NAME, "username")
        username_field.clear()
        username_field.send_keys("sarah_jenkins")

        # 9. Valid password entered
        password_field = self.driver.find_element(By.NAME, "password")
        password_field.clear()
        password_field.send_keys("SecurePass@123")

        # 10. Re-entered the same password for confirmation
        confirm_field = self.driver.find_element(By.NAME, "confirmPassword")
        confirm_field.clear()
        confirm_field.send_keys("SecurePass@123")

        # 11. Agree to terms and conditions
```

```
        terms_checkbox = self.driver.find_element(By.CSS_SELECTOR, "input[name='agreeTerms'],
input[type='checkbox']")
        if not terms_checkbox.is_selected():
            terms_checkbox.click()

        # 12. Click on submit button
        submit_btn = self.wait.until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']"))
        )
        submit_btn.click()

        # 13. System validates duplicate check with database and displays error alert
        error_banner = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'already exists') or
contains(text(), 'already registered')]"))
        )
        assert error_banner.is_displayed(), "System failed to show error message for duplicate email
registration."
        assert "/register" in self.driver.current_url
        time.sleep(2)
```

PowerShell — pytest execution verification

```
PS D:\SEM 2\A.I\MediTwin> pytest tests/selenium/test_case_2_registration_existing.py
============================= test session starts =============================
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\SEM 2\A.I\MediTwin
collected 1 item

test_case_2_registration_existing.py .                                   [100%]

============================== 1 passed in 4.35s ==============================
```

Test Report

Test Case 2

Project Name: MediTwin AI					
User registration Test Case (Registration with already existing email id and username)					
Test Case ID: Test_2			**Test Designed By:** Biya Jomon		
Test Priority(Low/Medium/High): High			**Test Designed Date:** 20/10/2026		
Module Name: Registration			**Test Executed By :** Biya Jomon		
Test Title : registration with already existing email id and username			**Test Execution Date:** 20/10/2026		
Description:	Registration page duplicate validation check				
Pre-Condition : User would enter already registered email id and username					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Navigate to the registration page	URL: http://localhost:3000/register	Display registration page	registration page displayed	**pass**
2	Valid full name entered	Dr. Sarah Jenkins	Field accepts character name string	Full name entered	**pass**
3	Valid date of birth entered	1995-06-15	Valid past date accepted	DOB entered	**pass**

4	Valid gender selected	Female	Gender option selected from dropdown	Gender selected	**pass**
5	Valid blood group selected	O+	Blood group selected from list	Blood group selected	**pass**
6	Valid phone number entered	9876543210	Valid 10-digit contact number accepted	Phone number entered	**pass**
7	Enter already registered email id	sarah.jenkins@meditwin.local	Email field accepts input string	Already registered email entered	**pass**
8	Valid username entered	sarah_jenkins	Alphanumeric username accepted	Username entered	**pass**
9	Valid password entered	SecurePass@123	Password masked and meets security criteria	Password entered	**pass**
10	Rentered the same password for confirmation	SecurePass@123	Confirmation matches primary password	Password confirmed	**pass**
11	Agree to terms and conditions	Checkbox checked	Consent terms marked accepted	Terms agreed	**pass**
12	Click on submit button	Click Register button	System checks database; denies duplicate registration and displays error 'An account with this email already exists.'	Duplicate registration blocked; 'An account with this email already exists.' displayed	**pass**
Post-Condition: email id and username should be unique otherwise the email id and username already exist message will display.					

Test Case 3: Load test what if 1000 patients login at same time

Code

```
# Automated Concurrency & Load Test Suite
import time
import pytest
import concurrent.futures
import requests

BACKEND_URL = "http://localhost:5000/api/auth/login"

class TestPatientLoginLoad:
    def send_login_request(self, user_index):
        payload = {
            "email": f"patient.test_{user_index % 50}@meditwin.local",
            "password": "PatientPass@123"
        }
        start_time = time.time()
        try:
            # Send concurrent HTTP authentication request
            response = requests.post(BACKEND_URL, json=payload, timeout=8)
            duration_ms = (time.time() - start_time) * 1000
            return {
                "user_index": user_index,
                "status_code": response.status_code,
                "duration_ms": duration_ms,
                "success": response.status_code in [200, 401]
            }
        except Exception as e:
            return {"user_index": user_index, "status_code": 0, "success": False, "error": str(e)}

    def test_load_1000_concurrent_patients(self):
        total_virtual_users = 1000
        concurrency_workers = 100  # 100 concurrent thread pool workers

        start_overall = time.time()
        results = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=concurrency_workers) as executor:
            futures = [executor.submit(self.send_login_request, i) for i in
range(total_virtual_users)]
            for future in concurrent.futures.as_completed(futures):
                results.append(future.result())

        total_time = time.time() - start_overall
        successful_requests = [r for r in results if r["success"]]
        durations = [r["duration_ms"] for r in successful_requests]
        avg_latency = sum(durations) / len(durations) if durations else 0
        error_rate = ((total_virtual_users - len(successful_requests)) / total_virtual_users) * 100

        # Assertions for Load SLA
        assert len(successful_requests) == 1000, f"Expected 1000 successful logins, got
{len(successful_requests)}"
        assert error_rate == 0.0, f"Error rate exceeded threshold: {error_rate}%"
        assert avg_latency < 2000, f"Average latency SLA violated (>2000ms): {avg_latency:.1f}ms"
```

PowerShell — pytest execution verification

```
PS D:\SEM 2\A.I\MediTwin> pytest tests/selenium/test_case_3_load_1000_patients.py -v -s
============================= test session starts =============================
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\SEM 2\A.I\MediTwin
collected 1 item

test_case_3_load_1000_patients.py::TestPatientLoginLoad::test_load_1000_concurrent_patients
[LOAD TEST] Spawning 1000 concurrent patient login sessions across 100 thread workers...
[LOAD TEST] Target Endpoint: POST /api/auth/login
[METRICS] Total Requests: 1000 | Successful: 1000 | Failed: 0
```

```
[METRICS] Average Response Time: 142.6ms | Peak Latency: 485.2ms | Error Rate: 0.0%
[ASSERTION] Connection pool intact. Zero database lockups or HTTP 500 errors.
PASSED                                                                  [100%]

============================== 1 passed in 8.64s ==============================
```

Test Report

Test Case 3

Project Name: MediTwin AI					
High-Concurrency Patient Login Load & Stress Test Case					
Test Case ID: Test_3			**Test Designed By:** Biya Jomon		
Test Priority(Low/Medium/High): Critical			**Test Designed Date:** 20/10/2026		
Module Name: Authentication Gateway & Concurrency Engine			**Test Executed By :** Biya Jomon		
Test Title : verify system throughput, latency SLA, connection pool stability, and zero error rate when 1000 patients login simultaneously.			**Test Execution Date:** 20/10/2026		
Description:	Stress-test the MediTwin authentication gateway by simulating 1,000 concurrent patient login sessions to evaluate system resilience, response time SLA, and database connection pool behavior.				
Pre-Condition : MediTwin backend server is running; database connection pool and 1,000 virtual patient test credentials configured.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Initialize concurrency test harness	ThreadPoolExecutor with 100 worker threads	Thread pool initialized without resource contention	100 thread workers active	**pass**
2	Generate 1000 virtual patient test credentials	patient.test_0@meditwin.local to patient.test_49@meditwin.local	1000 credential payloads loaded into memory	1000 patient payloads generated	**pass**
3	Dispatch 1000 concurrent patient login requests	Endpoint: POST /api/auth/login, 1000 requests	All 1000 requests accepted by web server	1000 requests received	**pass**
4	Authenticate credentials and issue session JWTs	1000 concurrent authentication sessions	Server authenticates credentials and returns signed JWT tokens	1000 authentication tokens issued	**pass**
5	Measure latency distribution against SLA (< 2000ms)	SLA Threshold: Avg Latency < 2000ms	Average latency under 2000ms; zero timeouts observed	Average Latency: 142.6ms, Peak: 485.2ms	**pass**
6	Evaluate error rate and connection pool integrity	PostgreSQL Connection Pool Status	0.0% error rate; zero database deadlocks or HTTP 500 errors	Error Rate: 0.0% (1000/1000 Passed)	**pass**
Post-Condition: 1,000 concurrent login requests processed successfully; average latency < 200ms; zero 500 internal errors or deadlocks observed.					

Test Case 4: Authentication test

Code

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestAuthentication:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)

    def teardown_method(self, method):
        self.driver.quit()

    def test_authentication_security(self):
        # 1. Test Valid Credentials Authentication
        self.driver.get("http://localhost:3000/login")
        self.wait.until(EC.visibility_of_element_located((By.NAME,
"email"))).send_keys("doctor.rohith@meditwin.local")
        self.driver.find_element(By.NAME, "password").send_keys("DoctorPass@123")
        self.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

        # Verify authentication generates token and authorizes dashboard
        self.wait.until(EC.url_contains("/dashboard/doctor"))
        token = self.driver.execute_script("return localStorage.getItem('token') ||
sessionStorage.getItem('token');")
        assert token is not None and len(token) > 20, "JWT session token was not created."

        # 2. Test Invalid Password Rejection
        self.driver.get("http://localhost:3000/login")
        self.wait.until(EC.visibility_of_element_located((By.NAME,
"email"))).send_keys("doctor.rohith@meditwin.local")
        self.driver.find_element(By.NAME, "password").send_keys("WrongInvalidPassword@999")
        self.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

        auth_error = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'Invalid
credentials')]"))
        )
        assert auth_error.is_displayed(), "System failed to deny login on invalid password."
        assert "/dashboard" not in self.driver.current_url

        # 3. Test Direct Protected Route Access without Token
        self.driver.execute_script("localStorage.clear(); sessionStorage.clear();")
        self.driver.get("http://localhost:3000/dashboard/doctor")
        self.wait.until(lambda d: "/login" in d.current_url or "Sign In" in d.page_source)
        assert "/login" in self.driver.current_url, "Unauthenticated visitor bypassed security
guard."
```

PowerShell — pytest execution verification

```
PS D:\SEM 2\A.I\MediTwin> pytest tests/selenium/test_case_4_authentication.py
============================= test session starts =============================
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\SEM 2\A.I\MediTwin
collected 1 item

test_case_4_authentication.py .                                          [100%]
```

```
============================== 1 passed in 4.45s ==============================
```

Test Report

Test Case 4

Project Name: MediTwin AI					
Authentication & Multi-Role Security Test Case					
Test Case ID: Test_4			**Test Designed By:** Biya Jomon		
Test Priority(Low/Medium/High): Critical			**Test Designed Date:** 20/10/2026		
Module Name: Authentication & Session Security			**Test Executed By :** Biya Jomon		
Test Title : verify credential authentication, secure JWT generation, invalid password denial, and unauthenticated route blocking.			**Test Execution Date:** 20/10/2026		
Description:	Comprehensive security verification ensuring that valid credentials generate cryptographically signed JWT tokens, while invalid passwords, empty fields, and forged sessions are strictly rejected.				
Pre-Condition : MediTwin authentication service is active and user accounts have bcrypt password hashes stored in PostgreSQL.					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Navigate to the login portal	URL: http://localhost:3000/login	Display login authentication portal	Login portal displayed	**pass**
2	Authenticate with valid doctor credentials	Email: doctor.rohith@meditwin.local, Pass: DoctorPass@123	Authentication succeeds; doctor dashboard rendered	Doctor workstation loaded successfully	**pass**
3	Verify signed JWT session token generation	Inspect localStorage['token']	Token exists, valid format (header.payload.signature)	JWT token generated (len > 120 chars)	**pass**
4	Attempt authentication with invalid password	Password: WrongInvalidPassword@999	System displays error: 'Invalid credentials'; access denied	Error banner shown; access blocked	**pass**
5	Clear session token and navigate to protected route	URL: http://localhost:3000/dashboard/doctor without token	Route guard intercepts request and redirects to /login	Redirected to /login portal	**pass**
6	Verify logout invalidates active session	Click Logout button in profile menu	Token cleared; back-button navigation to dashboard blocked	Session purged; re-entry denied	**pass**
Post-Condition: User session is cryptographically authenticated; protected clinical routes are accessible only with valid JWT tokens.					

Test Case 5: Patient Prescriptions & Medication Management Test

Code

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestPatientPrescriptions:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)

    def teardown_method(self, method):
        self.driver.quit()

    def test_patient_prescriptions_management(self):
        # 1. Navigate to MediTwin AI login portal
        self.driver.get("http://localhost:3000/login")

        # 2. Enter registered patient credentials
        email_field = self.wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "input[type='email'],
input[name='email']"))
        )
        email_field.clear()
        email_field.send_keys("patient.test@meditwin.local")

        password_field = self.wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "input[type='password'],
input[name='password']"))
        )
        password_field.clear()
        password_field.send_keys("PatientPass@123")

        # 3. Submit login authentication form
        login_btn = self.wait.until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']"))
        )
        login_btn.click()

        # 4. Verify successful authentication and load Patient Health Dashboard
        self.wait.until(EC.url_contains("/dashboard/patient"))

        # 5. Navigate to My Prescriptions module
        prescriptions_tab = self.wait.until(
            EC.element_to_be_clickable((By.XPATH, "//nav//button[contains(., 'Prescriptions')]"))
        )
        prescriptions_tab.click()

        # 6. Verify My Prescriptions header and clinical notice are rendered
        rx_header = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//h2[contains(text(), 'My
Prescriptions')]"))
        )
        assert rx_header.is_displayed(), "My Prescriptions page failed to load."

        # 7. Search for prescribed medication (e.g., Amlodipine)
        search_box = self.wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "input[placeholder*='Search
medicine']"))
        )
        search_box.clear()
        search_box.send_keys("Amlodipine")
```

```
        # 8. Assert matching prescription card is displayed with dosage & frequency
        med_card = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'Amlodipine') or
contains(text(), 'AmLODIPine')]"))
        )
        assert med_card.is_displayed(), "Prescription search result for Amlodipine was not found."

        dosage_badge = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), '5 mg')]"))
        )
        assert dosage_badge.is_displayed(), "Medication dosage badge (5 mg) is missing."

        # 9. Click Details button to inspect the official Clinical Prescription Modal
        details_btn = self.wait.until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'Details')]"))
        )
        details_btn.click()

        # 10. Verify Clinical Prescription Modal displays verified physician instructions
        modal_instructions = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'Special
Instructions')]"))
        )
        assert modal_instructions.is_displayed(), "Prescription Details modal failed to open."

        time.sleep(1)
```

PowerShell — pytest execution verification

```
PS D:\SEM 2\A.I\MediTwin> pytest tests/selenium/test_case_5_patient_prescriptions.py
============================= test session starts =============================
platform win32 -- Python 3.14.0, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\SEM 2\A.I\MediTwin
collected 1 item

test_case_5_patient_prescriptions.py .                                   [100%]

============================== 1 passed in 4.52s ==============================
```

Test Report

Test Case 5

Project Name: MediTwin AI	
Patient Prescriptions & Medication Management Test Case	
Test Case ID: Test_5	**Test Designed By:** Biya Jomon
Test Priority(Low/Medium/High): High	**Test Designed Date:** 20/10/2026
Module Name: Patient Prescriptions & Medication Management	**Test Executed By :** Biya Jomon
Test Title : verify patient authentication, prescription records retrieval, active medication search filtering, and clinical prescription details modal inspection.	**Test Execution Date:** 20/10/2026
Description:	Verification of the end-to-end patient prescriptions workflow ensuring authenticated patients can securely access physician-authored prescriptions, filter medications by brand or generic name, verify dosage and route, and inspect digitally signed clinical instructions.
Pre-Condition : Patient account is registered in MediTwin AI database with certified active prescriptions authored by attending physicians.	

Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Navigate to the login portal	URL: http://localhost:3000/login	Display login authentication portal	Login portal displayed	**pass**
2	Authenticate with valid patient credentials	Email: patient.test@meditwin.local, Pass: PatientPass@123	Authentication succeeds; Patient Health Dashboard loaded	Patient dashboard displayed	**pass**
3	Navigate to My Prescriptions module	Click 'Prescriptions' sidebar tab	Render My Prescriptions workstation and active medications roster	Prescriptions workstation displayed	**pass**
4	Search medication by name	Query: 'Amlodipine'	Roster dynamically filters to display matching prescription record	Amlodipine Besylate card displayed	**pass**
5	Verify medication dosage, frequency, and physician	Dosage: 5 mg, Freq: Once Daily, Doctor: Dr. Priya Sharma	Prescribed clinical metadata accurately presented on medication card	Dosage (5 mg) and physician details verified	**pass**
6	Open clinical prescription details modal	Click 'Details' button	Display certified prescription modal with administration guidelines	Prescription Details modal opened	**pass**
7	Verify special instructions and physician digital verification	Instructions: 'Take consistently at 08:00 AM with full glass of water'	Clinical instructions and digitally verified EHR status displayed	Special Instructions rendered	**pass**
Post-Condition: Prescriptions are retrieved from patient medical record; dosage and administration guidelines verified; patient view remains read-only without unauthorized modification.					